

Lab Report

Lab 1: System Call Implementation

GitHub: <https://github.com/RutanshS/CS202-Lab-1>

Video Demo: [Video Demo Link](#)

Subject: Advanced Operating Systems

Instructor: Prof. Cong Liu

Email: congl@ucr.edu

Due Date: 15th Oct 2025 11:59:59 PDT

Names: Rutansh Suthar, Akshat Shah, Kopal Chakravarty

Emails: rsuth004@ucr.edu, ashah004@ucr.edu,
kchak010@ucr.edu

Student ID: 862549045, 862477556, 862548292

Part-1 The sysinfo system call

In this part, we want to create a new system call **sysinfo** to provide user-level programs with specific information about a particular process which is currently executing. For this, we track new per-process data and securely pass this from the kernel to the user level.

Kernel Level Changes

1. kernel/syscall.h

Define new system call number for **sysinfo**. This is an identifier for the kernel and user space for the system call.

```
1 // .....
2 #define SYS_sysinfo 23
```

2. kernel/syscall.c

Here we register the new syscall. So using the extern keyword, we declared the **sys_sysinfo()** syscall function. Also updated the syscalls array to map **SYS_sysinfo** identifier i.e. 23 to **sys_sysinfo** function. Then we created a global counter named **syscall_count**, which is used to track the number of syscalls made since booting up the system. This is done by incrementing the counter whenever the **syscall()** function is successfully called. The value of this global counter is exposed using another function called **get_syscall_count()**.

```
1 // .....
2 extern uint64 sys_sysinfo(void); // sysinfo: declaration
3 // .....
4 static uint64 (*syscalls[])(void) = {
5 // .....
6 [SYS_sysinfo] sys_sysinfo, // sysinfo: syscall entry
7 };
8
9 uint syscall_count = 0; // sysinfo: syscall counter - stores total number of syscalls
10
11 void
12 syscall(void)
13 {
14     int num;
15     struct proc *p = myproc();
16
17     num = p->trapframe->a7;
18     if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
19         // Use num to lookup the system call function for num, call it,
20         // and store its return value in p->trapframe->a0
21         syscall_count++; // new addition
22         p->trapframe->a0 = syscalls[num]();
23     } else {
24         printf("%d %s: unknown sys call %d\n",
```

```

25         p->pid, p->name, num);
26     p->trapframe->a0 = -1;
27 }
28 }
29
30 // sysinfo: expose value of syscall counter
31 int
32 get_syscall_count(void)
33 {
34     return syscall_count;
35 }

```

3. kernel/proc.c

Added the count_active_processes() function that basically counts the number of functions that are active at that time. This is done by accessing the proc[] which has the list of all processes and count each of them that is not in the UNUSED state.

```

1 // .....
2 // sysinfo: return total active (ready, running, waiting and zombie) processes.
3 int
4 count_active_processes(void)
5 {
6     struct proc *p;
7     int count = 0;
8     for (p = proc; p < &proc[NPROC]; p++)
9     {
10         if (p -> state != UNUSED)
11         {
12             count++;
13         }
14     }
15     return count;
16 }

```

4. kernel/kalloc.c

Added the count_free_pages() function that basically counts the number of available memory pages by traversing the linked list kmem.freelist.

```

1 // .....
2 int
3 count_free_pages(void)
4 {
5     struct run *r;
6     int count = 0;
7

```

```

8   acquire(&kmem.lock);
9   r = kmem.freelist;
10  while(r) {
11      count++;
12      r = r->next;
13  }
14  release(&kmem.lock);
15
16  return count;
17 }

```

5. kernel/defs.h

Here we define the way that the helper functions are meant to be used. This information is used by sysproc.c to call these functions.

```

1  // .....
2  // kalloc.c
3  // .....
4  int          count_free_pages(void); // sysinfo
5  // .....
6  // proc.c
7  // .....
8  int          count_active_processes(void); // sysinfo
9  // .....
10 // syscall.c
11 // .....
12 int          get_syscall_count(void); // sysinfo
13 // .....

```

6. kernel/sysproc.c

This is the main logic of the new syscall. Whenever this syscall is performed, it gets the argument using argint and stores it in the variable param. Now based on the value of this param we call either of the 3 helper functions and return their values. If the argument given is invalid then we return -1 as specified.

```

1  // .....
2  // return either total running processes right now
3  // or total syscalls made since start
4  // or no. of free memory pages right now
5  uint64
6  sys_sysinfo(void)
7  {
8      int param;
9      argint(0, &param);
10
11      int result;

```

```

12     switch(param)
13     {
14         case 0:
15             return count_active_processes();
16         case 1:
17             return get_syscall_count();
18         case 2:
19             return count_free_pages();
20         default:
21             return -1;
22     }
23     return result;
24 }

```

User Level Changes

1. user/sysinfo_test.c

This is a test for checking the functionality of the sysinfo syscall. It calls the sysinfo command with all the 3 valid arguments and prints out the values.

```

1  #include "kernel/types.h"
2  #include "kernel/stat.h"
3  #include "user/user.h"
4
5  void print_sysinfo(void)
6  {
7      int n_active_proc, n_syscalls, n_free_pages;
8
9      n_active_proc = sysinfo(0);
10     n_syscalls = sysinfo(1);
11     n_free_pages = sysinfo(2);
12
13     printf("[sysinfo] active proc: %d, syscalls: %d, free pages: %d\n",
14           n_active_proc, n_syscalls, n_free_pages);
15 }
16
17 int main(void)
18 {
19     print_sysinfo();
20     exit(0);
21 }

```

2. user/user.h

In this file the sysinfo function is defined for the user to use in the test file.

```

1  \\ .....

```

```
2 int sysinfo(int); // sysinfo
3 \\ .....
```

3. user/usys.pl

This enables the usage of the sysinfo command for the user.

```
1 # .....
2 # sysinfo syscall for user
3 entry("sysinfo");
```

Makefile

Here we add the test file command.

```
1 # .....
2 # test is hello syscall test
3 # sysinfo_test is sysinfo syscall test
4 # .....
5     $U/_sysinfo_test\
6 # .....
```

Part-2 The procinfo system call

In this part, we want to create a new system call **procinfo** to provide user-level programs with specific information about a particular process which is currently executing. For this, we track new per-process data and securely pass this from the kernel to the user level.

0.1 Per-Process System Call Counts

0.1.1 Adding new system call

```
1 // .....
2 #define SYS_procinfo 24
```

The very first thing we do is to add a new system call for **procinfo** process in **kernel/syscall.h** file. So, we add **SYS_procinfo**.

0.1.2 Adding in syscall list

```
1 // .....
2 extern uint64 sys_procinfo(void);
```

```

3
4 static uint64 (*syscalls[])(void) = {
5 //...
6 [SYS_procinfo] sys_procinfo,
7 };

```

Next, we add the **procinfo** system call that we just defined in the syscall list, which is located in the **kernel/syscall.c** file. To do this, we first add `sys_procinfo` before the syscall list. Then, at the end of the list, we add `[SYS_procinfo] sys_procinfo`.

0.1.3 Adding function prototypes

```

1 // .....
2 uint64 sys_sysinfo(void);
3 uint64 sys_procinfo(void);

```

In the **kernel/defs.h** file we add the function prototype for our procinfo system call. We add **sys_procinfo** along with other prototypes in the `sysproc.c` section in the file.

0.1.4 Adding user entry

```

1 // .....
2 entry("sysinfo");
3 entry("procinfo");

```

In the user space we add the entry for **procinfo** so that kernel knows what to do when procinfo is called from user end. So in **user/usys.pl** we add `entry("procinfo")`.

Then in **user/user.h** file we define a new struct in order to define the information we want to store for a specific process. So we define

```

1 struct pinfo {
2     int ppid;
3     int syscall_count;
4     int page_usage;
5 };
6
7 //.....
8 int procinfo(struct pinfo*);

```

and then we add the system call below in system calls as **procinfo**.

0.1.5 Adding New syscall_counter variable

```
1 struct proc {
2     // ...
3     int syscall_count;
4 };
```

In **kernel/proc.h** file we have a struct defined for each particular process. In this struct, we add our own `syscall_counter` integer which will give us the number of system calls made by that single process running.

0.1.6 Modifying `allocproc` and `fork` functions

Now in **kernel/proc.c** we need to modify two functions i.e. `allocproc()` and `fork()`.

```
1     // in allocproc().....
2     // .....
3     p->syscall_count = 0;
4     // .....
```

- `allocproc()` function is the function that sets up the base structure for every new process with a clean state. So we initialize our `syscall_counter` here so that every process has their own counters from zero. So we add **`p -> syscall_counter = 0`** in the `allocproc()` function.

```
1     // in fork().....
2     // .....
3     np->syscall_count = p->syscall_count; // Copy the counter to the child
4     np->parent = p;
5     // .....
```

- `fork()` function is the function which copies the parent's state into their child's state. As we want our child process to have the counter of our parent we do that in `fork()` function. So we add **`np -> syscall_counter = p -> syscall_counter`**.

0.1.7 Incrementing counter in `syscall`

```
1     // in syscall().....
2     // .....
3     syscall_count++; // for sysinfo.
4     p->syscall_counter++; // for procinfo
5     // .....
```

In **kernel/syscall.c** we increment the `syscall_counter` we have defined in order to get the syscalls of a particular process.

0.1.8 Implementing the main logic of `procinfo` system call

```

1
2 struct pinfo {
3     int ppid;
4     int syscall_count;
5     int page_usage;
6 };
7
8 uint64
9 sys_procinfo(void)
10 {
11     uint64 user_addr;
12     struct pinfo kernel_pinfo;
13     struct proc *p = myproc();
14
15     argaddr(0, &user_addr);
16
17     kernel_pinfo.ppid = p->parent->pid;
18
19     kernel_pinfo.syscall_count = p->syscall_count - 1;
20
21     kernel_pinfo.page_usage = (p->sz + PGSIZE - 1) / PGSIZE;
22
23     if (copyout(p->pagetable, user_addr, (char *)&kernel_pinfo, sizeof(kernel_pinfo)) < 0) ←
24     {
25         return -1;
26     }
27     return 0;
28 }

```

Finally, in **kernel/sysproc.c**, we implement the main logic to get the information for a specific running process. We implement `pinfo` just like in **user/user.h** here as well, and then we implement `sys_procinfo(void){}` for the main logic. Then we add `user_addr`, and we make a temporary `pinfo` structure named `kernel_pinfo`. After that, we get the current process using `myproc()` and store it in `proc *p`. Then we retrieve the argument using `argaddr` and store it in the `user_addr` we defined.

Next, we get the parent process ID from the current process and store it inside `kernel_pinfo.ppid`. We then compute the system call count by subtracting 1 from the process's `syscall_counter` (to exclude the current call), and store it in `kernel_pinfo.syscall_count`. We also assign the pageusage to `kernel_pinfo.page_usage`. Finally, we copy our data from the temporary `kernel_pinfo` struct into the user-provided pointer, and if successful, we return.

This is how we can get all the information about one specific running process.

1 Contributions

- Study and In depth Research about Process and System Calls - Rutansh, Kopal, Akshat.
- Coding Contributions:

- Part-1 sysinfo system call active processes, total system calls and free memory pages - Rutansh.
 - Part-2 procinfo system call active processes, parent process ID and per process system calls - Akshat.
 - Part-2 procinfo system call memory page usage by process and free memory pages - Kopal.
- Lab Report - Rutansh and Kopal.
 - Lab Demo - Akshat.